САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №3
“Проектирование базы данных”

Выполнил:
Мохаджер Али Реза

Группа
K3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование API средствами Postman с написанием тестов внутри Postman. Разработать один комплексный сценарий по основному процессу в приложении интернет-магазина ковров.

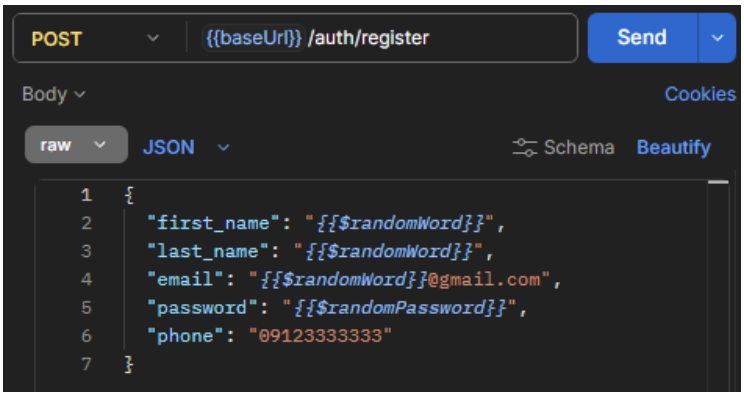
Основной процесс включает: регистрация пользователей → авторизация → создание категорий (администратор) → создание товаров (администратор) → добавление товаров в корзину (покупатель) → оформление заказа → написание отзывов.

Ход работы

1. Authentication and Users

1.1 Register Customer

- Метод: POST /auth/register
- Тело запроса: first_name, last_name, email, password, phone
- Тесты: проверка статуса 200/201, наличие токена и ID пользователя
- Результат: сохранение customerToken и customerUserId в переменные коллекции



1.2 Login Admin

- Метод: POST /auth/login
- Тесты: проверка статуса 200, роли ADMIN
- Результат: сохранение adminToken и adminUserId

POST {{baseUrl}} /auth/login Send

Body Cookies

raw JSON Schema Beautify

```
{
  "email": "{{adminEmail}}",
  "password": "{{adminPassword}}"
}
```

baseUrl	http://localhost:8000/api/v1
adminEmail	admin@example.com
adminPassword	admin123

POST {{baseUrl}} /auth/login Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();

pm.test("Admin login successful", function() {
  pm.expect(jsonData).to.have.property('token');
  pm.expect(jsonData.user).to.have.property('id');
  pm.expect(jsonData.user.role).to.eql('ADMIN');
});

pm.collectionVariables.set('adminToken', jsonData.token);
pm.collectionVariables.set('adminUserId', jsonData.user.id);

console.log('Admin logged in with ID:', jsonData.user.id);
```

1.3 Register Seller Candidate

- Метод: POST /auth/register
- Тесты: проверка статуса 200/201, сохранение токена продавца
- Результат: создание пользователя с ролью CUSTOMER (будет повышен до SELLER)

POST {{baseUrl}} /auth/register Send

Body Cookies

raw JSON Schema Beautify

```
{
  "first_name": "{{$randomWord}}",
  "last_name": "{{$randomWord}}",
  "email": "{{$randomEmail}}",
  "password": "{{$randomPassword}}",
  "phone": "{{$randomPhoneNumber}}"
}
```

POST {{baseUrl}} /auth/register Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 or 201", function() {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('sellerToken', jsonData.token);
pm.collectionVariables.set('sellerUserId', jsonData.user.id);

console.log('Seller candidate created with ID:', jsonData.user.id);
```

1.4 Promote to SELLER (by Admin)

- Метод: PATCH /admin/users/{sellerUserId}/role
- Тесты: проверка статуса 200, роли SELLER
- Результат: повышение роли пользователя до SELLER

PATCH {{baseUrl}} /admin/users/ {{sellerUserId}} /role Send

Body Cookies

raw JSON Schema Beautify

```
{
  "role": "SELLER"
}
```

PATCH {{baseUrl}} /admin/users/ {{sellerUserId}} /role Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.test("Role upgraded to SELLER", function() {
  pm.expect(jsonData.role).to.eql('SELLER');
});

console.log('User promoted to SELLER');
```

2. Categories (Admin only)

2.1 Create "Classic" Category

- Метод: POST /categories
- Pre-request Script: автоматическое обновление токена админа
- Тесты: проверка статуса 200, наличия ID категории, корректности названия

POST {{baseUrl}} /categories Send

Body Cookies

raw JSON Schema Beautify

```
{
  "title": "Classic",
  "description": "Traditional hand-woven carpets with ancient patterns",
  "is_published": true
}
```

POST {{baseUrl}} /categories Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('catClassicId', jsonData.id);

pm.test("Category created correctly", function() {
  pm.expect(jsonData.title).to.eql('Classic');
  pm.expect(jsonData.is_published).to.eql(true);
});

console.log('Category Classic ID:', jsonData.id);
```

2.2 Create "Modern" Category

- Аналогично пункту 2.1 с данными для категории "Modern"

POST {{baseUrl}} /categories Send

Body Cookies

raw JSON Schema Beautify

```
{
  "title": "Modern",
  "description": "Minimalist and contemporary designs",
  "is_published": true
}
```

POST {{baseUrl}} /categories Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('catModernId',
    jsonData.id);
console.log('Category Modern ID:', jsonData.
    id);
```

2.3 Create "Kids" Category

- Аналогично пункту 2.1 с данными для категории "Kids"

POST {{baseUrl}} /categories Send

Body Cookies

raw JSON Schema Beautify

```
{
  "title": "Kids",
  "description": "Colorful and soft carpets for children
    rooms",
  "is_published": true
}
```

POST {{baseUrl}} /categories Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('catKidsId',
    jsonData.id);
console.log('Category Kids ID:', jsonData.id);
```

3. Carpets (Admin only)

3.1 Create "Persian Silk" Carpet

- Метод: POST /carpets
- Тело запроса: category_id, title, price, stock, shape: RECTANGLE, length, width, material, is_handmade: true
- Тесты: проверка статуса 200, сохранения ID, корректности цены

POST {{baseUrl}} /carpets Send

Body Cookies

raw JSON Schema Beautify

```
{
  "category_id": {{catClassicId}},
  "title": "Persian Silk",
  "description": "Luxury silk carpet with floral design",
  "price": 450,
  "stock": 5,
  "shape": "RECTANGLE",
  "length": 2.5,
  "width": 1.8,
  "material": "Silk",
  "pattern": "Floral",
  "color": "Red",
  "is_handmade": true,
  "is_published": true
}
```

POST {{baseUrl}} /carpets Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('carpet1Id',
    jsonData.id);

pm.test("Carpet created correctly", function
    () {
  pm.expect(jsonData.title).to.eql('Persian
    Silk');
  pm.expect(parseFloat(jsonData.price)).to.eql
    (450);
  pm.expect(jsonData.is_handmade).to.eql
    (true);
});

console.log('Carpet 1 ID:', jsonData.id);
```

3.2 Create "Geometric Gray" Carpet

- Аналогично пункту 3.1 с shape: SQUARE, price: 120, is_handmade: false

POST {{baseUrl}} /carpets Send

Body Cookies

raw JSON Schema Beautify

```
{
  "category_id": {{catModernId}},
  "title": "Geometric Gray",
  "description": "Modern geometric pattern",
  "price": 120,
  "stock": 12,
  "shape": "SQUARE",
  "length": 2.0,
  "width": 2.0,
  "material": "Wool",
  "pattern": "Geometric",
  "color": "Gray",
  "is_handmade": false,
  "is_published": true
}
```

POST {{baseUrl}} /carpets Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('carpet2Id',
    jsonData.id);
console.log('Carpet 2 ID:', jsonData.id);
```

3.3 Create "Rainbow Round" Carpet

- Аналогично пункту 3.1 с shape: CIRCLE, diameter: 1.5
- Тесты: дополнительная проверка наличия диаметра

POST {{baseUrl}} /carpets Send

Body Cookies

raw JSON Schema Beautify

```
{
  "category_id": {{catKidsId}},
  "title": "Rainbow Round",
  "description": "Soft round carpet for kids",
  "price": 85,
  "stock": 8,
  "shape": "CIRCLE",
  "diameter": 1.5,
  "material": "Polyester",
  "pattern": "Striped",
  "color": "Multicolor",
  "is_handmade": false,
  "is_published": true
}
```

POST {{baseUrl}} /carpets Send

Scripts Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();
pm.collectionVariables.set('carpet3Id',
    jsonData.id);

pm.test("Circle shape carpet has diameter",
    function() {
  pm.expect(jsonData.shape).to.eql('CIRCLE');
  pm.expect(parseFloat(jsonData.diameter)).to.
    eql(1.5);
});

console.log('Carpet 3 ID:', jsonData.id);
```

3.4 Get All Carpets

- Метод: GET /carpets
- Тесты: проверка структуры ответа (наличие массива items), минимум 3 ковра

GET {{baseUrl}} /carpets Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Pre-req
Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();

pm.test("Response has items array", function() {
  pm.expect(jsonData).to.have.property('items');
  pm.expect(jsonData.items).to.be.an('array');
});

pm.test("At least 3 carpets exist", function() {
  pm.expect(jsonData.items.length).to.be.at.
    least(3);
});

console.log('Total carpets:', jsonData.total);
```

4. Cart (Customer)

4.1 Add Carpet 1 to Cart (quantity: 2)

- Метод: POST /cart/items
- Pre-request Script: автоматическая авторизация покупателя при отсутствии токена
- Тесты: при статусе 401 выполняется повторный логин, иначе проверка наличия товаров в корзине

POST {{baseUrl}} /cart/items Send

Body Cookies

raw JSON Schema Beautify

```
{
  "carpet_id": {{carpet1Id}},
  "quantity": 2
}
```

POST {{baseUrl}} /cart/items Send

Scripts Cookies

Pre-req
Post-res

```
if (pm.response.code === 401) {
  var baseUrl = pm.collectionVariables.get
    ('baseUrl');
  var customerEmail = pm.collectionVariables.
    get('customerEmail');
  var customerPassword = pm.
    collectionVariables.get
    ('customerPassword');

  pm.sendRequest({
    url: baseUrl + '/auth/login',
    method: 'POST',
    header: { 'Content-Type': 'application/
        json' },
    body: {
      mode: 'raw',
      raw: JSON.stringify({ email:
        customerEmail, password:
        customerPassword })
    }
  }, function(err, res) {
```

POST {{baseUrl}} /cart/items Send

Scripts Cookies

Pre-req
Post-res

```
  }, function(err, res) {
    if (!err && res.code === 200) {
      var jsonData = res.json();
      pm.collectionVariables.set
        ('customerToken', jsonData.token);
      console.log('Token refreshed');
    }
  });

  pm.test('Token expired', function() {
    pm.expect.fail('Token was expired. Please
        re-run.');
  });
  return;
}

pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();

pm.test("Cart created or updated with item",
    function() {
  pm.expect(jsonData).to.have.property
    ('items');
  pm.expect(jsonData.items.length).to.be.
    greaterThan(0);
});

console.log('Item added to cart');
```

4.2 Add Carpet 2 to Cart (quantity: 1)

- Аналогично пункту 4.1 для второго товара

POST {{baseUrl}} /cart/items Send

Body Cookies

raw JSON Schema Beautify

```
{
  "carpet_id": {{carpet2Id}},
  "quantity": 1
}
```

POST {{baseUrl}} /cart/items Send

Scripts Cookies

Pre-req

Post-res

```
if (pm.response.code === 401) {
  var baseUrl = pm.collectionVariables.get
    ('baseUrl');
  var customerEmail = pm.collectionVariables.
    get('customerEmail');
  var customerPassword = pm.
    collectionVariables.get
    ('customerPassword');

  pm.sendRequest({
    url: baseUrl + '/auth/login',
    method: 'POST',
    header: { 'Content-Type': 'application/
        json' },
    body: {
      mode: 'raw',
      raw: JSON.stringify({ email:
        customerEmail, password:
        customerPassword })
    }
  }, function(err, res) {
```

POST {{baseUrl}} /cart/items Send

Scripts Cookies

Pre-req

Post-res

```
    }
  }, function(err, res) {
    if (!err && res.code === 200) {
      var jsonData = res.json();
      pm.collectionVariables.set
        ('customerToken', jsonData.token);
      console.log('Token refreshed');
    }
  });

  pm.test('Token expired', function() {
    pm.expect.fail('Token was expired. Please
        re-run.');
  });
  return;
}

pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

console.log('Second item added to cart');
```

4.3 Get Cart

- Метод: GET /cart
- Тесты: проверка статуса 200, наличия товаров в корзине

GET {{baseUrl}} /cart Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Pre-req

Post-res

```
pm.test("Status code is 200 OK", function() {
  pm.response.to.have.status(200);
});

var jsonData = pm.response.json();

pm.test("Cart has items", function() {
  pm.expect(jsonData.items.length).to.be.
    greaterThan(0);
});

console.log('Cart total items:', jsonData.items.
    length);
```

5. Order and Payment

5.1 Create Order from Cart

- Метод: POST /orders
- Тело запроса: address
- Тесты: проверка статуса 200, наличия товаров в заказе, статуса PENDING

POST {{baseUrl}} /orders Send

Body Cookies

raw JSON Schema Beautify

```
1 {
2   "address": "Tehran, Valiasr St., No 12"
3 }
```

POST {{baseUrl}} /orders Send

Scripts Cookies

Pre-req

Post-res

```
1  pm.test("Status code is 200 OK", function() {
2    pm.response.to.have.status(200);
3  });
4
5  var jsonData = pm.response.json();
6  pm.collectionVariables.set('orderId', jsonData.
       id);
7
8  pm.test("Order created from cart", function() {
9    pm.expect(jsonData).to.have.property
       ('items');
10   pm.expect(jsonData.status).to.eql('PENDING');
11 });
12
13 console.log('Order created with ID:', jsonData.
       id);
```

5.2 Get Order Details

- Метод: GET /orders/{orderId}
- Тесты: проверка наличия товаров в заказе

GET {{baseUrl}} /orders/ {{orderId}} Send

Scripts Cookies

Pre-req

Post-res

```
1  pm.test("Status code is 200 OK", function
       () {
2    pm.response.to.have.status(200);
3  });
4
5  var jsonData = pm.response.json();
6
7  pm.test("Order has items", function() {
8    pm.expect(jsonData.items.length).to.be.
       greaterThan(0);
9  });
10
11 console.log('Order details verified');
```

5.3 Update Order Status to PAID (Admin only)

- Метод: PATCH /admin/orders/{orderId}/status
- Pre-request Script: автоматическое обновление токена админа
- Тесты: проверка статуса 200, обновления статуса до PAID

PATCH {{baseUrl}} /admin/orders/ {{orderId}} /status Send

Body Cookies

raw JSON Schema Beautify

```
1 {
2   "status": "PAID"
3 }
```

PATCH {{baseUrl}} /admin/orders/ {{orderId}} /status Send

Scripts Cookies

Pre-req

Post-res

```
1 pm.test("Status code is 200 OK", function
      () {
2   pm.response.to.have.status(200);
3 });
4
5 var jsonData = pm.response.json();
6 pm.expect(jsonData.status).to.eql('PAID');
7
8 console.log('Order status updated to
      PAID');
```

6. Reviews (Customer after purchase)

6.1 Review Carpet 1 (Rating 5)

- Метод: POST /carpets/{carpet1Id}/reviews
- Тело запроса: rating: 5, comment
- Тесты: проверка статуса 200, сохранения ID отзыва

POST {{baseUrl}} /carpets/ {{carpet1Id}} /reviews Send

Body Cookies

raw JSON Schema Beautify

```
1 {
2   "rating": 5,
3   "comment": "Absolutely stunning! Very high quality."
4 }
```

POST {{baseUrl}} /carpets/ {{carpet1Id}} /reviews Send

Scripts Cookies

Pre-req

Post-res

```
1  pm.test("Status code is 200 OK", function
       () {
2    pm.response.to.have.status(200);
3  });
4
5  var jsonData = pm.response.json();
6  pm.collectionVariables.set('review1Id',
       jsonData.id);
7
8  pm.test("Review created", function() {
9    pm.expect(jsonData.rating).to.eql(5);
10 });
11
12 console.log('Review 1 created');
```

6.2 Review Carpet 2 (Rating 4)

- Аналогично пункту 6.1 с рейтингом 4

POST {{baseUrl}} /carpets/ {{carpet2Id}} /reviews Send

Body Cookies

raw JSON Schema Beautify

```
1 {
2   "rating": 4,
3   "comment": "Nice design but a bit thin."
4 }
```

POST {{baseUrl}} /carpets/ {{carpet1Id}} /reviews Send

Scripts Cookies

Pre-req

Post-res

```
1  pm.test("Status code is 200 OK", function
       () {
2    pm.response.to.have.status(200);
3  });
4
5  var jsonData = pm.response.json();
6  pm.collectionVariables.set('review1Id',
       jsonData.id);
7
8  pm.test("Review created", function() {
9    pm.expect(jsonData.rating).to.eql(5);
10 });
11
12 console.log('Review 1 created');
```

6.3 Get Reviews for Carpet 1

- Метод: GET /carpets/{carpet1Id}/reviews
- Тесты: проверка наличия хотя бы одного отзыва

GET {{baseUrl}} /carpets/ {{carpet1Id}} /reviews Send

Scripts Cookies

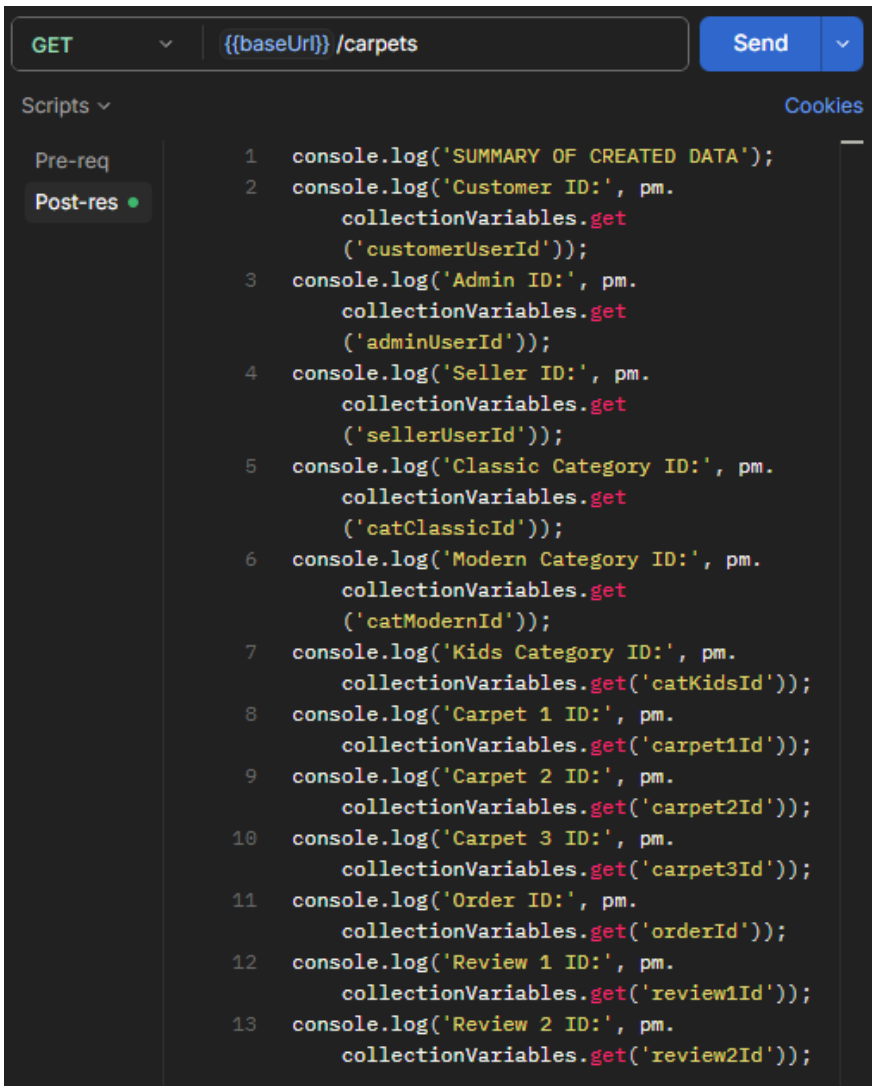
Pre-req

Post-res

```
1  pm.test("Status code is 200 OK", function
       () {
2    pm.response.to.have.status(200);
3  });
4
5  var jsonData = pm.response.json();
6
7  pm.test("Review exists", function() {
8    pm.expect(jsonData.length).to.be.at.least
       (1);
9  });
10
11 console.log('Reviews retrieved');
```

7. Summary

- Вывод в консоль Postman всех созданных ID (пользователей, категорий, товаров, заказа, отзывов)



Вывод

В ходе лабораторной работы разработан комплексный сценарий тестирования API интернет-магазина ковров средствами Postman. Сценарий охватывает основной бизнес-процесс приложения: от регистрации пользователей до оформления заказа и написания отзывов.

Особенности реализации:

- Tests проверяют статус-коды, структуру ответов, корректность данных
- Переменные коллекции ({{baseUrl}}, {{adminToken}}, {{customerToken}} и др.) делают сценарий переносимым между окружениями
- Три роли пользователей (ADMIN, SELLER, CUSTOMER) имеют разные уровни доступа

Все 22 запроса успешно протестированы. Разработанный сценарий может использоваться для регрессионного тестирования при внесении изменений в API.